

UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE LOS VALLES



Maestría en Ingeniería de Software

Course: Administración de la configuración del software

Professor: Omar Alí Zatarain Durán

Essay - Elements of Software Configuration Management

Student: Maribel Navarro Jaquiz

Date: September 11, 2026

Elements of Software Configuration Management

The goal of the software developer is product integrity. Product integrity is the set of attributes that characterize a product:

- that fulfills user functional needs;
- that can easily and completely be traced through its life cycle;
- that meets specified performance criteria;
- whose cost expectations are met;
- whose delivery expectations are met.

Software is not just a set of computer programs. It also includes the documentation required to define, develop, and maintain those programs.

To build a software project, most developers structure themselves into three basic discipline sets: project management, development, and product assurance.

Project management shows what is happening in the project, makes sure the organization consistently develops products with integrity, and determines the relative allocation of resources between the other two.

The development disciplines include analysis, design, engineering, production (coding), test (unit/subsystem), installation, documentation, training, and maintenance.

The product assurance disciplines give project management visibility into the development process. They include configuration management, quality assurance, validation and verification, and test and evaluation. Using them well is basic to the success of the project, because they provide the technical checks and balances over the product being developed.

Elements of SCM

Software Configuration Identification

To manage the development of a system, we must define the components of its baseline and also the changes to those components. The baselines and the changes together specify the system evolution.

A baseline is like a snapshot of the system components at a given point in time. Each update is a set of changes made on top of that snapshot. The role of software configuration identification is to provide labels for those snapshots and for the whole sequence.

A baseline is characterized by two labels: one identifies the baseline itself, and the other identifies an update to that baseline. An update is a baseline plus a set of changes that have been incorporated into it.

The most elementary entity of software configuration identification is the software configuration item (SCI). A baseline appears as a set of SCI's, and the SCI's within a baseline are related to one another via a tree-like hierarchy. SCI's are not only code files. They also include requirements documents, user manuals, design specifications, and test plans. Each baseline and each update will exist in one or more forms, such as a design document, source code on a disk, or executing object code.

Each baseline and its associated updates show the evolution of the software during each of its life cycle stages. Identifying them provides an explicit documentation trail linking all stages of the software life cycle. With this trail, the software developer can assess the integrity of his product, and the software buyer can assess the integrity of the product he is paying for.

Software Configuration Control

The evolution of a software system is the development of baselines and the incorporation of a series of changes into the baselines. Some changes affect existing baselines. Other changes happen early in the system life cycle and can affect baselines that do not yet exist. The role of software configuration control is to provide the administrative mechanism for precipitating, preparing, evaluating, and approving or disapproving all change proposals throughout the system life cycle.

It focuses on managing changes to SCI's, existing or to be developed, in all of their representations. This process involves three basic ingredients. The first is documentation for formally precipitating and defining a proposed change to a software system. The major control document is the Engineering Change Proposal (ECP), which contains a description of the proposed change, the rationale for the change, identification of affected baselines and SCI's, and specification of cost and schedule impacts. The second is an organizational body for formally evaluating and approving or disapproving a proposed change, the Configuration Control Board (CCB). The third is procedures for controlling changes to a software system. Many automated tools support this process, and they are generically referred to as program support libraries (PSL's). A PSL provides a centralized and readily available repository for authoritative versions of each component of a software system.

Software Configuration Auditing

It provides the mechanism for determining the degree to which the current state of the software system mirrors the software system pictured in baseline and requirements documentation.

Software configuration auditing serves two purposes:

- Configuration verification: ensuring that what is intended for each software configuration item as specified in one baseline is actually achieved in the succeeding baseline or update.

- Configuration validation: ensuring that the SCI configuration solves the right problem, that is, that customer needs are satisfied.

Software auditing is intended to increase software visibility and to establish traceability throughout the life cycle. It also makes disconnects visible. These disconnects include requirements not satisfied in the audited product and extraneous features observed in the product, that is, features for which no stated requirement exists. With this visibility, project management can evaluate the integrity of the software product being developed, resolve issues that may have been raised by the audit, and correct defects in the development process.

Software Configuration Status Accounting

It is the mechanism for maintaining a record of how the system has evolved and where the system is at any time relative to what appears in published baseline documentation and written agreements. It maintains a history of how the system moved from one state to another, and it records the activity associated with identification, control, and auditing.

The scope of software configuration status accounting encompasses the recording and reporting of:

- the time at which each representation of a baseline and update came into being, and the time at which each software configuration item came into being;
- descriptive information about each SCI;
- engineering change proposal status (approved, disapproved, awaiting action), and descriptive information about each ECP;
- change status, and descriptive information about each change;
- status of technical and administrative documentation associated with a baseline or update;
- deficiencies in a to-be-established baseline uncovered during a configuration audit.

Because of its large data input and output requirements, status accounting is generally supported in part by automated processes such as the PSL. Data are collected and organized for input to a computer, and reports giving the status of entities are compiled and generated by the computer.

References:

- [1] E. H. Bersoff, "Elements of software configuration management," *IEEE Trans. Softw. Eng.*, vol. SE-10, no. 1, pp. 79–87, Jan. 1984.